Problem Sheet 1 (CSE/Inf/Geo)

Deadline: Tuesday, 14.10.2025

1.1. Consider the approximation of the integral $\int_0^1 f(x)$ by the trapezoidal rule

$$\int_0^1 f(x)\,dx \approx T(h) := \sum_{i=0}^{N-1} \frac{h}{2}\left(f(x_i) + f(x_{i+1})\right), \qquad x_i = ih, \qquad h = \frac{1}{N}$$

as well as the box rule

$$\int_0^1 f(x)\,dx \approx R(h) := \sum_{i=0}^{N-1} hf(x_i), \qquad x_i = ih, \qquad h = \frac{1}{N}.$$

a) Write one program that, given N and a function f, returns $T(h)$ and one that returns $R(h)$ with $h = 1/N$.

b) For $f(x) = e^x$ and $N(i) = 2^i$, $i = 1, \ldots, 10$, set $h(i) = 1/N(i)$. For these 10 values of h, compute the actual errors $\texttt{err}(i) = \int_0^1 e^x\,dx - T(h(i)) = e - 1 - T(h(i))$. Plot the (absolute value) of the true error in a "log-log" plot. That is, in `matlab` notation with the vectors h and `err` (both of length 10)

$$\texttt{loglog}(h, \texttt{abs}(\texttt{err}))$$

Does the true error behave like Ch^2?

Do the same thing for the box rule as well. What error behaviour do you see?

Remark: In `python` loglog plotting is realized with `matplotlib.pyplot.loglog` and the array limits have to be adapted since arrays start at 0 in `python`.

1.2. Again, consider the approximation of the integral $\int_0^1 f(x)$ by the trapezoidal rule $T(h)$ from exercise 1.1. The aim is to design an error estimator as it was done for the "box scheme" in the lecture.

a) For the error estimator, make the ansatz

$$\int_0^1 f(x)\,dx - T(h) \stackrel{!}{=} Ch^2$$

and compute C from (calculated) $T(h)$ and $T(h/2)$. The error estimator is then $E(h) = Ch^2$.

b) Modify your program from exercise 1.1(a) such that it also computes the error estimator $E(h)$.

For the test-case in exercise 1.1(b) do also compute the ratios $\texttt{r}(i) := \texttt{err}(i)/E(h(i))$ of the true error to estimated error. Plot the ratio `r` in a semilogarithmic plot versus h. That is, in `matlab` notation with the vectors h and `r` (both of length 10)

$$\texttt{semilogx}(h, \texttt{r})$$

1.3. Let the data $(0, 0)$, $(1, 2)$, $(4, 8)$ be given.

a) Write down the (quadratic) interpolating polynomial p in the Lagrange form, i.e., $p(x) = \sum_{i=0}^{2} f_i \ell_i(x)$.

b) The quadratic interpolating polynomial $p(x) = a_0 + a_1 x + a_2 x^2$ can also be determined by solving a linear system of equations for the coefficients a_0, a_1, a_2. Formulate the system and solve for the a_i.

c) Check your approximation $p(2)$ using the `matlab` or `numpy` routines $\texttt{polyfit}(\mathbf{x}, \mathbf{f}, n)$ and `polyval`. Note that `polyfit` also returns the coefficients a_i of part b). Compare.

Note: The use of `polyfit` and `polyval` in the context of polynomial interpolation should rather be viewed as a "quick and dirty" method since `polyfit` is based on polynomial approximation in the monomial basis $\{1, x, x^2, \ldots, x^n\}$ and therefore numerically unsuitable for large/largish n.

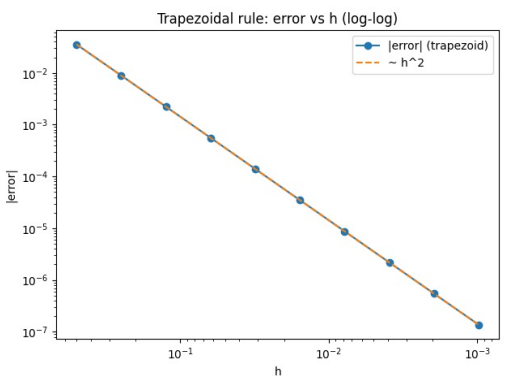
1 1a

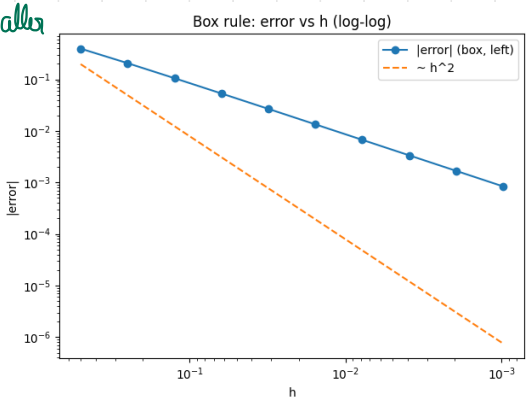
```python
import numpy as np
import matplotlib.pyplot as plt

def trapezoid(f, N, a=0.0, b=1.0):
    h = (b - a) / N
    x = np.linspace(a, b, N + 1)
    return h * (0.5 * f(x[0]) + f(x[1:-1]).sum() + 0.5 * f(x[-1])), h

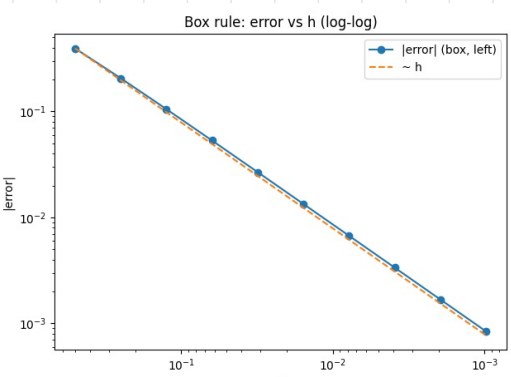
def box_rule(f, N, a=0.0, b=1.0):
    h = (b - a) / N
    x = a + h * np.arange(0, N)
    return h * np.sum(f(x)), h
```

b, Ch = how fast is the error getting smaller





Yes, the true error behaves like Ch^2 as you can see in the plot



The Box rule: The error behaves like Ch (first order). Since e^x is increasing, the sum underestimates, so $(e-1) - R(h) > 0$

1.2 a

$$\int_0^1 f(x)dx - T(h) \approx Ch^2$$

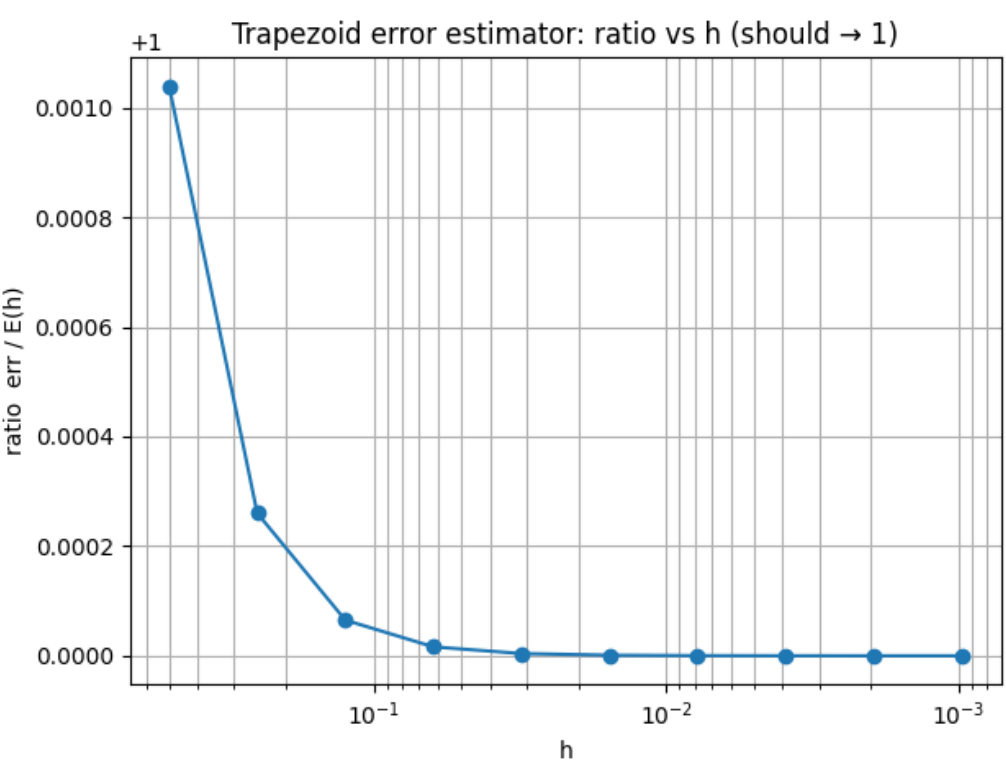
$$\int_0^1 f(x)dx - T\left(\frac{h}{2}\right) \approx C\left(\frac{h}{2}\right)^2 = \frac{1}{4}Ch^2$$

Now we subtract the 2 equations:

$$T\left(\frac{h}{2}\right) - T(h) \approx \left(\frac{1}{4}Ch^2\right) - Ch^2 = -\frac{3}{4}Ch^2 \implies C \approx \frac{4}{3} \cdot \frac{T\left(\frac{h}{2}\right) - T(h)}{h^2}$$

Answer: $E(h) = Ch^2 \approx \frac{4}{3}T\left(\frac{h}{2}\right) - T(h)$

b,



1.3 a

$$l_0(x) = \frac{(x-1)(x-4)}{(0-1)(0-4)} = \frac{(x-1)(x-4)}{4}$$

$$l_1(x) = \frac{(x-0)(x-4)}{(1-0)(1-4)} = -\frac{x(x-4)}{3}$$

$$l_2(x) = \frac{(x-0)(x-1)}{(4-0)(4-1)} = \frac{x(x-1)}{12}$$

$$p(x) = \sum_{i=0}^{2} f_i l_i(x) = 0 \cdot \frac{(x-1)(x-4)}{4} + 2\left(-\frac{x(x-4)}{3}\right) + 8 \cdot \left(\frac{x(x-1)}{12}\right)$$

$$= -\frac{2}{3}(x^2 - 4x) + \frac{2}{3}(x^2 - x) = \frac{2}{3}(-x^2 + 4x + x^2 - x)$$

$$= 2x$$

b)

Vandermonde matrix

$$\begin{cases} (x=0) & a_0 = 0 \\ (x=1) & a_0 + a_1 + a_2 = 2 \\ (x=4) & a_0 + 4a_1 + 16a_2 = 8 \end{cases} \iff \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 4 & 16 \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 2 \\ 8 \end{pmatrix}$$

(i) $a_0 = 0$

(ii) $a_1 + a_2 = 2$

(iii) $4a_1 + 16a_2 = 8$

1. I want to eliminate a_1
   - multiply ii by 4
     ↳ $4a_1 + 4a_2 = 8$
   - subtract this from iii
     ↳ $(4a_1 + 16a_2) - (4a_1 + 4a_2) = 8 - 8 \Rightarrow 12a_2 = 0$

$a_2 = 0$

$a_1 + 0 = 2 \Rightarrow a_1 = 2$

$a_0 = 0, a_1 = 2, a_2 = 0$

c)

```python
import numpy as np

x = np.array([0.0, 1.0, 4.0])
y = np.array([0.0, 2.0, 8.0])

coef_polyfit = np.polyfit(x, y, deg=2)

p2_polyval = np.polyval(coef_polyfit, 2.0)

V = np.column_stack([np.ones_like(x), x, x**2])
a0, a1, a2 = np.linalg.solve(V, y)      # expect a0=0, a1=2, a2=0

print("polyfit coeffs [a2,a1,a0]:", coef_polyfit)
print("polyval at x=2:", p2_polyval)
print("system coeffs  [a2,a1,a0]:", [a2, a1, a0])
print("system p(2):", a0 + a1*2 + a2*(2**2))
```

```
polyfit coeffs [a2,a1,a0]: [6.05225017e-16 2.00000000e+00 5.12790050e-16]
polyval at x=2: 4.0000000000000036
system coeffs  [a2,a1,a0]: [np.float64(-0.0), np.float64(2.0), np.float64(0.0)]
system p(2): 4.0
```

✓

1.4. **a)** Write a routine that has as input the vectors $\mathbf{x}$ and $\mathbf{f}$ of knots and data (both vectors have the length $n+1$) and returns the value of the interpolating polynomial at the point 0, i.e., $p(0)$ and the polynomial p of degree n satisfies $p(\mathbf{x}_i) = \mathbf{f}_i$ for all i. To simplify matters, you may use `polyfit` and `polyval`.

b) Based on a), write a function whose input is a function handle f and integers m, $n \in \mathbb{N}_0$. The output is a matrix $N \in \mathbb{R}^{(m+1)\times(n+1)}$ with entries $N_{j,k} := p_{j,k}(0)$, where $j = 0, \ldots, m$, $k = 0, \ldots, n$ and $p_{j,k}$ is the polynomial of degree k that interpolates the data $(h_{j+\ell}, f(h_{j+\ell}))$, $\ell = 0, \ldots, k$, where $h_i := 2^{-i}$.

Remark: This matrix N is the so-called *Neville*-scheme for interpolation although it is usually set up differently. (This will be done later in class.)

c) Apply your routine of b) to the function

$$f(h) = \frac{e^h - 1}{h}$$

and select $m = 10$ and $n = 2$. You expect the columns of N to converge to $\lim_{h\to 0} f(h) = 1$. To check that, plot (in `matlab` notation)

$$\texttt{loglog}(h(1:m{+}1), \texttt{abs}(N(:,1){-}1), \quad h(1:m{+}1), \texttt{abs}(N(:,2){-}1), \quad h(1:m{+}1), \texttt{abs}(N(:,3){-}1))$$

What convergence behavior do you observe for these 3 graphs?

1.5. Let $f(x) = \frac{x^2}{1+x^2}$. Use the knots $x_0 = 0$, $x_1 = 1$, $x_2 = 2$ and values $f_i = f(x_i)$.

a) Compute the divided differences $f[x_i]$, $f[x_0, x_1]$, $f[x_1, x_2]$, and $f[x_0, x_1, x_2]$ and use them to produce a quadratic interpolation polynomial in Newton form.

b) Add the data point $(3, f(3))$ and update your quadratic interpolation polynomial to obtain a cubic interpolation polynomial in Newton form.

c) Use both interpolation polynomials to compute approximations to $\int_0^3 f(x)dx$ and calculate the errors.

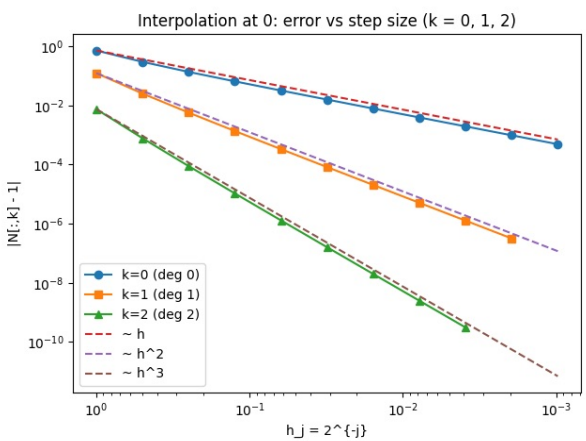
1.4 a

```python
def interp_at_zero(x, f):
    x = np.asarray(x, dtype=float)
    f = np.asarray(f, dtype=float)
    deg = len(x) - 1
    coefs = np.polyfit(x, f, deg=deg)
    return np.polyval(coefs, 0.0)
```

b)

```python
def neville_matrix(f, m, n):
    h = 2.0 ** (-np.arange(0, m + 1))
    N = np.zeros((m + 1, n + 1), dtype=float)
    for j in range(m + 1):
        for k in range(n + 1):
            if j + k > m:
                N[j, k] = np.nan
            else:
                x_nodes = h[j:j + k + 1]
                y_nodes = f(x_nodes)
                N[j, k] = interp_at_zero(x_nodes, y_nodes)
    return h, N
```

c)



All three curves converge to 1 as $h \to 0$, but with different orders

On the log-log plot the slopes are $\sim 1, \sim 2, \sim 3$ for $k = 0, 1, 2$

So the observed convergence is first order for $k=0$, second order for $k=1$, and third order for $k=2$

1.5 a

$f(x) = \frac{x^2}{1+x^2}$ $\quad f(0)=0 \quad f(1)=\frac{1}{2} \quad f(2)=\frac{4}{5} \quad f(3)=\frac{9}{10}$

$f[x_0] = 0$ $\quad f[x_0,x_1] = \frac{f(1)-f(0)}{1-0} = \frac{1}{2}$

$f[x_1] = \frac{1}{2}$

$f[x_2] = \frac{4}{5}$ $\quad f[x_1,x_2] = \frac{f(2)-f(1)}{2-1} = \frac{3}{10}$

$f[x_0,x_1,x_2] = \frac{f[x_1,x_2]-f[x_0,x_1]}{2-0} = \frac{\frac{3}{10}-\frac{1}{2}}{2} = -\frac{1}{10}$

Newton form:

$p_2(x) = f[x_0] + f[x_0,x_1](x-x_0) + f[x_0,x_1,x_2](x-x_0)(x-x_1) = \frac{1}{2}x - \frac{1}{10}(x^2-x)$

$= \frac{1}{2}x - \frac{1}{10}x^2 + \frac{1}{10}x = (\frac{1}{2}+\frac{1}{10})x - \frac{1}{10}x^2 = \underline{\frac{3}{5}x - \frac{1}{10}x^2}$

b, enhance with $x_3 = 3$

$f[x_2,x_3] = \frac{f(3)-f(2)}{3-2} = \frac{\frac{9}{10}-\frac{4}{5}}{1} = \frac{9}{10}-\frac{8}{10} = \frac{1}{10}$

$f[x_1,x_2,x_3] = \frac{f[x_2,x_3]-f[x_1,x_2]}{x_3-x_1} = \frac{\frac{1}{10}-\frac{3}{10}}{3-1} = -\frac{1}{10}$

$f[x_0,x_1,x_2,x_3] = \frac{f[x_1,x_2,x_3]-f[x_0,x_1,x_2]}{x_3-x_0} = \frac{(-\frac{1}{10})-(-\frac{1}{10})}{3-0} = \frac{0}{3} = \underline{0}$

$p_3(x) = p_2(x) = \frac{3}{5}x - \frac{1}{10}x^2$

c,

$\int_0^3 p(x)\,dx = \int_0^3 (\frac{3}{5}x - \frac{1}{10}x^2)\,dx = \underline{\frac{9}{5} = 1{,}8} \rightarrow$ approximation

$\int_0^3 \frac{x^2}{1+x^2}\,dx = 3 - \arctan 3 \Rightarrow$ Theorem 1.15 $f(x) - p(x)$

$\Rightarrow$ error $= \left|\frac{9}{5} - (3-\arctan 3)\right| = \left|\arctan 3 - \frac{6}{5}\right| \approx \underline{4{,}9 \cdot 10^{-2}}$